

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

on

OPERATING SYSTEMS

(23CS4PCOPS)

Submitted by

SHAIKH UZAIR AHMED (1BM23CS307)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2025 to June-2025

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by **SHAIKH UZAIR AHMED (1BM23CS307)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2024. The Lab report has been approved as it satisfies the academic requirements in respect of a **OPERATING SYSTEMS - (23CS4PCOPS)** work prescribed for the said degree.

Amruta
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one) a) FCFS b) SJF	5.
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	7.
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms a) Rate- Monotonic	10.
4.	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	12.
5.	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance.	16.
6.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	18.
7.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	21.
8.	Write a C program to simulate the following file allocation strategies. a) Sequential b) Indexed c) Linked	27.

Course Outcome

CO1	Apply the different concepts and functionalities of Operating System
CO2	Analyze various Operating system strategies and techniques
CO3	Demonstrate the different functionalities of Operating System
CO4	Conduct practical experiments to implement the functionalities of Operating system

Lab Program 1:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one)

- a) FCFS
- b) SJF

```
#include<stdio.h>
struct FCFS
{
    int pid;
    int at;
    int bt;
    int ct;
    int tat;
    int wt;
};
void sort(struct FCFS p[], int n){
    int min = p[0].at;
    int temp = 0;
    for (int i = 0; i < n; i++)
    {
        min=p[i].at;
        for (int j = i; j < n; j++)
        {
            if (p[j].at<min)
            {
                temp = p[i].at;
                p[i].at = p[j].at;
                p[j].at = temp;
                temp = p[j].bt;
                p[j].bt = p[i].bt;
                p[i].bt = temp;
                temp = p[i].pid;
                p[i].pid = p[j].pid;
                p[j].pid = temp;
            }
        }
    }
}

int main(){
    int n,c=0;
    double avg_tat=0.0,ttat=0.0,avg_wt=0.0,twt=0.0;
    printf("Enter Number of Process :");
    scanf("%d",&n);
    struct FCFS p[n];
    for (int i = 0; i < n; i++)
    {
        printf("Enter details for Process %d:\n ",(i+1));
        p[i].pid = (i+1);
        printf("Enter Process %d arrival time: \n",(i+1));
```

```

scanf("%d",&p[i].at);
printf("Enter Process %d burst time: \n",(i+1));
scanf("%d",&p[i].bt);
}

sort(p,n);
for(int i=0;i<n;i++)
{
    if(c>=p[i].at){
        c+=p[i].bt;
    }else
    {
        c+= p[i].at-p[i-1].ct+p[i].bt;
    }
    p[i].ct = c;
}

for(int i=0;i<n;i++){
    p[i].tat= p[i].ct-p[i].at;
}
for(int i=0;i<n;i++){
    p[i].wt= p[i].tat-p[i].bt;
}

printf("FCFS Sheduling: \n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",p[i].pid,p[i].at,p[i].bt,p[i].ct,p[i].tat,p[i].wt);
}
for (int i = 0; i < n; i++){
    ttat+=p[i].tat;
    twt+=p[i].wt;
}
avg_tat = ttat/(double)n;
avg_wt = twt/(double)n;
printf("Average Turn Around Time :%lf",avg_tat);
printf("Average Waiting Time :%lf",avg_wt);
}

```

Output:

```

Enter number of processes:
3
Enter arrival times:
0 1 2
Enter burst times:
5 3 1
FCFS Scheduling:
PID AT  BT  CT  TAT WT
1   0   5   5   5  0
2   1   3   8   7  4
3   2   1   9   7  6

Average turnaround time:6.333333ms

Average waiting time:3.333333ms

```

Lab Program 2:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories –system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include<stdio.h>
#include<stdlib.h>

#define MAX 10
#define MAXNAME 20

int c = 0;

typedef struct{
    int id, AT, BT, TAT, WT, CT;
    char name[MAXNAME];
}Process;

void print_process_data(Process q[], int n) {
    printf("PID\tName\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%s\t%d\t%d\t%d\t%d\t%d\n", q[i].id, q[i].name, q[i].AT, q[i].BT, q[i].CT, q[i].TAT, q[i].WT);
    }

    int ttat = 0, twt = 0;
    for (int i = 0; i < n; i++) {
        ttat += q[i].TAT;
        twt += q[i].WT;
    }

    double avg_tat = (double)ttat / n;
    double avg_wt = (double)twt / n;

    printf("Average Turn-around time: %lf\n", avg_tat);
    printf("Average Waiting time: %lf\n", avg_wt);
}

void sorted(Process q[], int n) {
    int i, j;
    for (i = 1; i < n; i++) {
        for (j = 0; j < n - i; j++) {
            if (q[j].AT > q[j + 1].AT || (q[j].AT == q[j + 1].AT && q[j].id > q[j + 1].id)) {
                Process temp = q[j];
                q[j] = q[j + 1];
                q[j + 1] = temp;
            }
        }
    }
}
```

```

void FCFS(Process q[], int n){
    for(int i=0; i<n; i++)
    {
        if (i == 0) {
            c = q[i].AT + q[i].BT;
        } else if (c >= q[i].AT) {
            c += q[i].BT;
        } else {
            c = q[i].AT + q[i].BT;
        }
        q[i].CT = c;
    }

    for(int i=0; i<n; i++){
        q[i].TAT = q[i].CT - q[i].AT;
        q[i].WT = q[i].TAT - q[i].BT;
    }

    print_process_data(q, n);
}

void input_process(Process q[], int *n)
{
    printf("Enter Number of Processes : \n");
    scanf("%d", n); // Fix here

    for (int i = 0; i < *n; i++) // Use *n instead of n
    {
        printf("\nEnter details for Process %d:\n ", (i+1));
        q[i].id = i+1;
        printf("\nEnter the Mode of Process\n");
        scanf("%s", q[i].name);
        printf("\nEnter Process %d arrival time:\n", (i+1));
        scanf("%d", &q[i].AT);
        printf("\nEnter Process %d burst time: \n", (i+1));
        scanf("%d", &q[i].BT);
    }
}

void main()
{
    Process SysQ[MAX], UserQ[MAX];
    int SystemCount, UserCount;

    printf("Enter Details for System Processes : \n");
    input_process(SysQ, &SystemCount);
    printf("Enter Details for User Processes : \n");
    input_process(UserQ, &UserCount);

    sorted(SysQ, SystemCount);
    sorted(UserQ, UserCount); // Fix: Sort UserQ instead of repeating SysQ
}

```



```

printf("Scheduling System Processes : \n");
FCFS(SysQ, SystemCount); // Fix: Call FCFS for SysQ

printf("Scheduling User Processes : \n");
FCFS(UserQ, UserCount);
}

```

Output:

```

Scheduling System Processes :
PID   Name   AT   BT   CT   TAT   WT
2     sysb   0    4    4    4     0
1     sysa   1    2    6    5     3
3     sysc   6    7   13    7     0
Average Turn-around time: 5.333333
Average Waiting time: 1.000000
Scheduling User Processes :
PID   Name   AT   BT   CT   TAT   WT
1     usera   1    2    3    2     0
2     userb   4    4    8    4     0
3     userc   9    9   18    9     0
Average Turn-around time: 5.000000
Average Waiting time: 0.000000

```

Lab Program 3:

Write a C program to simulate Real-Time CPU Scheduling algorithms:

Rate- Monotonic

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_TASKS 10

typedef struct {
    int id, period, RT, CT; // RT = Remaining Time, CT = Completion Time (Execution Time)
} Process;

// Sort tasks based on their period (shorter period means higher priority in RMS)
void sorted(Process p[], int n) {
    int i, j;
    for (i = 1; i < n; i++) {
        for (j = 0; j < n - i; j++) {
            if (p[j].period > p[j + 1].period || (p[j].period == p[j + 1].period && p[j].id > p[j + 1].id)) {
                Process temp = p[j];
                p[j] = p[j + 1];
                p[j + 1] = temp;
            }
        }
    }
}

// Rate Monotonic Scheduling
void RMS(Process p[], int n, int totaltime) {
    sorted(p, n); // Sort tasks based on their period

    // Initialize remaining times (RT) and set all tasks to their execution time
    for (int i = 0; i < n; i++) {
        p[i].RT = p[i].CT; // Set RT to the execution time initially
    }

    for (int t = 0; t < totaltime; t++) {
        int executed = 0;

        // Check for each task and run it if ready
        for (int i = 0; i < n; i++) {
            // Release task at the start of its period (every t % period == 0) and if RT > 0
            if (t % p[i].period == 0 && p[i].RT > 0) {
                p[i].RT = p[i].CT; // Reset remaining time when a task finishes its period
                printf("Time %d: Task %d released\n", t, p[i].id);
            }

            // If time `t` is a multiple of the task's period and the task still has remaining time
            if (t % p[i].period == 0 && p[i].RT > 0) {
                printf("Time %d: Executing Task %d\n", t, p[i].id);
                p[i].RT--; // Decrease remaining time of the task
            }
        }
    }
}
```

```

        executed = 1;

        if (p[i].RT == 0) { // Task is finished after execution
            printf("Time %d: Task %d completed\n", t + 1, p[i].id);
        }
        break; // Task executed, break out of loop
    }
}

// If no task was executed, CPU is idle
if (!executed) {
    printf("Time %d: Idle\n", t);
}
}
}

int main() {
    Process p[MAX_TASKS] = {
        {1, 3, 2, 2}, // Task 1: period=3, RT=2, CT=2
        {2, 5, 2, 2}, // Task 2: period=5, RT=2, CT=2
        {3, 7, 2, 2} // Task 3: period=7, RT=2, CT=2
    };

    int numofTask = 3;
    int TotalTime = 15; // Total simulation time

    RMS(p, numofTask, TotalTime);

    return 0;
}

```

Output:

```

PS C:\Users\Admin\Desktop\RMS> cd "C:\Users\Admin\Desktop\RMS\" ; if ($?) { gcc RMS.c -o RMS } ; if ($?) { .\RMS }
Time 0: Executing Task 1
Time 1: Idle
Time 2: Idle
Time 3: Executing Task 1
Time 4: Task 1 completed
Time 4: Idle
Time 5: Executing Task 2
Time 6: Task 1 released
Time 6: Executing Task 1
Time 7: Executing Task 3
Time 8: Idle
Time 9: Executing Task 1
Time 10: Task 1 completed
Time 10: Executing Task 2
Time 11: Task 2 completed
Time 11: Idle
Time 12: Task 1 released
Time 12: Executing Task 1
Time 13: Idle
Time 14: Executing Task 3
Time 15: Task 3 completed

```

Lab Program 4:

Write a C program to simulate:

- a) Producer-Consumer problem using semaphores.
- b) Dining-Philosopher's problem

```
//Producer-Consumer
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h> // for sleep()

#define SIZE 5

int buffer[SIZE];
int in = 0, out = 0;
int empty = SIZE; // semaphore: counts empty slots
int full = 0;     // semaphore: counts full slots
int mutex = 1;    // binary semaphore for mutual exclusion

void wait(int *s) {
    while (*s <= 0); // busy wait
    (*s)--; // decrement semaphore value
}

void signal(int *s) {
    (*s)++; // increment semaphore value
}

void produce(int item) {
    wait(&empty); // wait for an empty slot
    wait(&mutex); // enter critical section

    buffer[in] = item;
    printf("Produced: %d at %d\n", item, in);
    in = (in + 1) % SIZE;

    signal(&mutex); // leave critical section
    signal(&full);  // signal that there is a full slot
}

void consume() {
    wait(&full); // wait for a full slot
    wait(&mutex); // enter critical section

    int item = buffer[out];
    printf("Consumed: %d from %d\n", item, out);
    out = (out + 1) % SIZE;

    signal(&mutex); // leave critical section
    signal(&empty); // signal that there is an empty slot
}

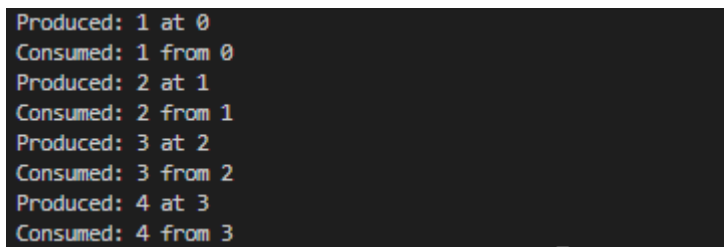
int main() {
    int item = 1;
```

```
// Simulate producer and consumer running automatically
while (1) {
    produce(item); // produce an item
    item++;        // increment item for next production
    sleep(1);      // simulate time taken to produce an item

    consume();     // consume an item
    sleep(1);      // simulate time taken to consume an item
}

return 0;
}
```

Output:



```
Produced: 1 at 0
Consumed: 1 from 0
Produced: 2 at 1
Consumed: 2 from 1
Produced: 3 at 2
Consumed: 3 from 2
Produced: 4 at 3
Consumed: 4 from 3
```

```

//Philosophers' Problem
#include <stdio.h>

#define N 5

int forks[N] = {1, 1, 1, 1, 1}; // binary semaphores for forks
int mutex = 1; // for critical section

void wait(int *s) {
    while (*s <= 0);
    (*s)--;
}

void signal(int *s) {
    (*s)++;
}

void pickup(int i) {
    wait(&mutex);

    if (forks[i] == 1 && forks[(i + 1) % N] == 1) {
        wait(&forks[i]);
        wait(&forks[(i + 1) % N]);
        printf("Philosopher %d picks up forks %d and %d and starts EATING.\n", i, i, (i + 1) % N);
    } else {
        printf("Philosopher %d can't eat now. One or both forks not available.\n", i);
    }

    signal(&mutex);
}

void putdown(int i) {
    wait(&mutex);

    signal(&forks[i]);
    signal(&forks[(i + 1) % N]);
    printf("Philosopher %d puts down forks %d and %d and starts THINKING.\n", i, i, (i + 1) % N);

    signal(&mutex);
}

void showForks() {
    printf("Fork status: ");
    for (int i = 0; i < N; i++) {
        printf("%d ", forks[i]);
    }
    printf("\n");
}

int main() {
    int choice, id;
    while (1) {
        printf("\n1. Pickup fork\n2. Putdown fork\n3. Show fork status\n4. Exit\nChoose: ");
        scanf("%d", &choice);
    }
}

```

```

switch (choice) {
    case 1:
        printf("Enter philosopher ID (0-4): ");
        scanf("%d", &id);
        if (id >= 0 && id < N)
            pickup(id);
        else
            printf("Invalid ID\n");
        break;
    case 2:
        printf("Enter philosopher ID (0-4): ");
        scanf("%d", &id);
        if (id >= 0 && id < N)
            putdown(id);
        else
            printf("Invalid ID\n");
        break;
    case 3:
        showForks();
        break;
    case 4:
        return 0;
    default:
        printf("Invalid choice\n");
}
}
}
}

```

Output:

```

1. Pickup fork
2. Putdown fork
3. Show fork status
4. Exit
Choose: 3
Fork status: 1 1 1 1 1

1. Pickup fork
2. Putdown fork
3. Show fork status
4. Exit
Choose: 1
Enter philosopher ID (0-4): 2
Philosopher 2 picks up forks 2 and 3 and starts EATING.

1. Pickup fork
2. Putdown fork
3. Show fork status
4. Exit
Choose: 1
Enter philosopher ID (0-4): 1
Philosopher 1 can't eat now. One or both forks not available.

1. Pickup fork
2. Putdown fork
3. Show fork status
4. Exit
Choose: 3
Fork status: 1 1 0 0 1

1. Pickup fork
2. Putdown fork
3. Show fork status
4. Exit
Choose: 2
Enter philosopher ID (0-4): 2
Philosopher 2 puts down forks 2 and 3 and starts THINKING.

1. Pickup fork
2. Putdown fork
3. Show fork status
4. Exit
Choose: 4

```

Lab Program 5:

Write a C program to simulate:

- a) Bankers' algorithm for the purpose of deadlock avoidance.

```
#include <stdio.h>
```

```
int main() {
    int n, m, i, j, k;
    printf("Enter the number of processes and resources : ");
    scanf("%d %d", &n, &m);

    int allo[n][m], max[n][m], need[n][m], avail[m];
    printf("Enter the allo matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++)
            scanf("%d", &allo[i][j]);

    printf("Enter the maximum demand matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
            need[i][j] = max[i][j] - allo[i][j];
        }

    printf("Enter the avail resources:\n");
    for (i = 0; i < m; i++)
        scanf("%d", &avail[i]);

    int finished[n], safeSequence[n], count = 0;
    for (i = 0; i < n; i++) finished[i] = 0;

    while (count < n) {
        int found = 0;
        for (i = 0; i < n; i++) {
            if (!finished[i]) {
                for (j = 0; j < m; j++)
                    if (need[i][j] > avail[j]) break;
                if (j == m) {
                    for (k = 0; k < m; k++)
                        avail[k] += allo[i][k];
                    safeSequence[count++] = i;
                    finished[i] = 1;
                    found = 1;
                }
            }
        }
        if (!found) {
            printf("System is in an unsafe state.\n");
            return 0;
        }
    }

    printf("System is in a safe state.\nSafe sequence: ");
```



```
    for (i = 0; i < n; i++) printf("%d ", safeSequence[i]);  
    printf("\n");  
    return 0;  
}
```

Output:

```
Enter number of processes: 5  
Enter number of resources: 3  
Enter allocation matrix (5 x 3):  
0 1 0  
2 0 0  
3 0 2  
2 1 1  
0 0 2  
Enter max matrix (5 x 3):  
7 5 3  
3 2 2  
9 0 2  
2 2 2  
4 3 3  
Enter available resources (3 values): 3 3 2  
System is in a safe state.  
Safe sequence: P1 P3 P4 P0 P2
```

Lab Program 6:

Write a C program to simulate the following contiguous memory allocation techniques.

- a) Worst-fit
- b) Best-fit
- c) First-fit

```
#include <stdio.h>
#include <stdlib.h>

void first_fit(int block_sizes[], int m, int process_sizes[], int n) {
    int allocation[n];

    // Initially, no process is allocated memory
    for (int i = 0; i < n; i++) {
        allocation[i] = -1;
    }

    // Assign memory to processes using First Fit
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (block_sizes[j] >= process_sizes[i]) {
                allocation[i] = j;
                block_sizes[j] -= process_sizes[i]; // Reduce the available block size
                break;
            }
        }
    }

    // Print the allocation results
    printf("\nFirst-Fit Allocation:\n");
    for (int i = 0; i < n; i++) {
        if (allocation[i] != -1) {
            printf("Process %d allocated to Block %d\n", i + 1, allocation[i] + 1);
        } else {
            printf("Process %d could not be allocated memory\n", i + 1);
        }
    }
}

void best_fit(int block_sizes[], int m, int process_sizes[], int n) {
    int allocation[n];

    // Initially, no process is allocated memory
    for (int i = 0; i < n; i++) {
        allocation[i] = -1;
    }

    // Assign memory to processes using Best Fit
    for (int i = 0; i < n; i++) {
        int best_idx = -1;
        int min_diff = 9999; // Initialize to a large number

        for (int j = 0; j < m; j++) {
            // Check if the block can accommodate the process
```

```

        if (block_sizes[j] >= process_sizes[i]) {
            int diff = block_sizes[j] - process_sizes[i];
            // Check if this block is the best fit
            if (diff < min_diff) {
                min_diff = diff;
                best_idx = j;
            }
        }
    }

    if (best_idx != -1) {
        allocation[i] = best_idx;
        block_sizes[best_idx] -= process_sizes[i]; // Reduce the available block size
    }
}

// Print the allocation results
printf("\nBest-Fit Allocation:\n");
for (int i = 0; i < n; i++) {
    if (allocation[i] != -1) {
        printf("Process %d allocated to Block %d\n", i + 1, allocation[i] + 1);
    } else {
        printf("Process %d could not be allocated memory\n", i + 1);
    }
}
}

int main() {
    int m, n;

    // Input the number of memory blocks and processes
    printf("Enter the number of memory blocks: ");
    scanf("%d", &m);
    int block_sizes[m];

    printf("Enter the sizes of the memory blocks:\n");
    for (int i = 0; i < m; i++) {
        printf("Block %d size: ", i + 1);
        scanf("%d", &block_sizes[i]);
    }

    printf("Enter the number of processes: ");
    scanf("%d", &n);
    int process_sizes[n];

    printf("Enter the sizes of the processes:\n");
    for (int i = 0; i < n; i++) {
        printf("Process %d size: ", i + 1);
        scanf("%d", &process_sizes[i]);
    }

    // Simulate First-Fit Allocation
    first_fit(block_sizes, m, process_sizes, n);
}

```

```

// Reset memory blocks for Best-Fit
int block_sizes_copy[m];
for (int i = 0; i < m; i++) {
    block_sizes_copy[i] = block_sizes[i];
}

// Simulate Best-Fit Allocation
best_fit(block_sizes_copy, m, process_sizes, n);

return 0;
}

```

Output:

```

Enter the number of memory blocks: 5
Enter the sizes of the memory blocks:
Block 1 size: 100
Block 2 size: 500
Block 3 size: 200
Block 4 size: 300
Block 5 size: 600
Enter the number of processes: 4
Enter the sizes of the processes:
Process 1 size: 212
Process 2 size: 417
Process 3 size: 112
Process 4 size: 426

First-Fit Allocation:
Process 1 allocated to Block 2
Process 2 allocated to Block 5
Process 3 allocated to Block 2
Process 4 could not be allocated memory

Best-Fit Allocation:
Process 1 allocated to Block 4
Process 2 could not be allocated memory
Process 3 allocated to Block 2
Process 4 could not be allocated memory

```

Lab Program 7:

Write a C program to simulate page replacement algorithms.

- a) FIFO
- b) LRU
- c) Optimal

a) FIFO

```
#include <stdio.h>
```

```
int main() {
    int frames, pages, pageFaults = 0, current = 0;
    int referenceString[100], frame[100]; // Max size to prevent runtime issues

    printf("Enter number of frames: ");
    scanf("%d", &frames);
    printf("Enter number of pages: ");
    scanf("%d", &pages);
    printf("Enter reference string: ");
    for (int i = 0; i < pages; i++) {
        scanf("%d", &referenceString[i]);
    }

    // Initialize frames
    for (int i = 0; i < frames; i++) {
        frame[i] = -1;
    }

    for (int i = 0; i < pages; i++) {
        int found = 0;
        for (int j = 0; j < frames; j++) {
            if (frame[j] == referenceString[i]) {
                found = 1;
                break;
            }
        }
        if (!found) {
            frame[current] = referenceString[i];
            current = (current + 1) % frames;
            pageFaults++;
        }
        printf("%d: ", referenceString[i]);
    }
    for (int k = 0; k < frames; k++) {
        printf("%d ", frame[k]);
    }
    printf("\n");

    printf("Total Page Faults: %d\n", pageFaults);
    return 0;
}
```

Output:

```
Enter number of frames: 3
Enter number of pages: 5
Enter reference string: 1 2 3 1 4
1: 1 -1 -1
2: 1 2 -1
3: 1 2 3
1: 1 2 3
4: 4 2 3
Total Page Faults: 4
```

b) LRU

```
#include<stdio.h>
```

```
int main() {
    int frames, pages, pageFaults = 0;
    int referenceString[100], frame[100]; // Max size to prevent runtime issues
    int recent[100]; // To track the order of pages for LRU

    printf("Enter number of frames: ");
    scanf("%d", &frames);
    printf("Enter number of pages: ");
    scanf("%d", &pages);
    printf("Enter reference string: ");
    for (int i = 0; i < pages; i++) {
        scanf("%d", &referenceString[i]);
    }

    // Initialize frames to -1 (indicating empty frames)
    for (int i = 0; i < frames; i++) {
        frame[i] = -1;
        recent[i] = -1; // Initialize recent access time
    }

    // Process each page in the reference string
    for (int i = 0; i < pages; i++) {
        int found = 0;

        // Check if the page is already in one of the frames
        for (int j = 0; j < frames; j++) {
            if (frame[j] == referenceString[i]) {
                found = 1; // Page hit, no page fault
                // Update recent access to indicate it's the most recently used
                recent[j] = i;
                break;
            }
        }

        // If the page is not found, we need to replace a page
        if (!found) {
            int emptyFrame = -1;

            // Check for empty frames and place the new page there if available
            for (int j = 0; j < frames; j++) {
                if (frame[j] == -1) {
                    emptyFrame = j; // Find the first empty frame
                    break;
                }
            }

            if (emptyFrame != -1) {
                // Place the page in the empty frame
                frame[emptyFrame] = referenceString[i];
                recent[emptyFrame] = i; // Mark the access time as the current index
            }
        }
    }
}
```

```

        pageFaults++; // Increment page fault
    } else {
        // If no empty frame, we need to replace the least recently used page
        int lruIndex = 0;
        for (int j = 1; j < frames; j++) {
            if (recent[j] < recent[lruIndex]) {
                lruIndex = j; // Find the least recently used page
            }
        }

        // Replace the least recently used page
        frame[lruIndex] = referenceString[i];
        recent[lruIndex] = i; // Update the access time for the replaced page
        pageFaults++; // Increment page fault
    }
}

// Display the current state of the frames
printf("%d: ", referenceString[i]);
for (int k = 0; k < frames; k++) {
    printf("%d ", frame[k]);
}
printf("\n");
}

// Output total page faults
printf("Total Page Faults: %d\n", pageFaults);
return 0;
}

```

Output:

```

Enter number of frames: 3
Enter number of pages: 20
Enter reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
7: 7 -1 -1
0: 7 0 -1
1: 7 0 1
2: 2 0 1
0: 2 0 1
3: 2 0 3
0: 2 0 3
4: 4 0 3
2: 4 0 2
3: 4 3 2
0: 0 3 2
3: 0 3 2
2: 0 3 2
1: 1 3 2
2: 1 3 2
0: 1 0 2
1: 1 0 2
7: 1 0 7
0: 1 0 7
1: 1 0 7
Total Page Faults: 12

```


c)Optimal

```
#include<stdio.h>
```

```
int main() {
    int frames, pages, pageFaults = 0;
    int referenceString[100], frame[100]; // Max size to prevent runtime issues

    printf("Enter number of frames: ");
    scanf("%d", &frames);
    printf("Enter number of pages: ");
    scanf("%d", &pages);
    printf("Enter reference string: ");
    for (int i = 0; i < pages; i++) {
        scanf("%d", &referenceString[i]);
    }

    // Initialize frames to -1 (indicating empty frames)
    for (int i = 0; i < frames; i++) {
        frame[i] = -1;
    }

    // Process each page in the reference string
    for (int i = 0; i < pages; i++) {
        int found = 0;

        // Check if the page is already in one of the frames
        for (int j = 0; j < frames; j++) {
            if (frame[j] == referenceString[i]) {
                found = 1; // Page hit, no page fault
                break;
            }
        }

        // If the page is not in any of the frames, replace a page using the OPT algorithm
        if (!found) {
            int index = -1;

            // First check for empty frames and put the new page there if available
            for (int j = 0; j < frames; j++) {
                if (frame[j] == -1) { // Find an empty frame to fill
                    index = j;
                    frame[index] = referenceString[i]; // Place the page in the empty frame
                    pageFaults++; // Increment page fault when a new page is loaded
                    break; // Exit as soon as we place the page in an empty frame
                }
            }

            // If no empty frame was found, use the OPT algorithm to find the page to replace
            if (index == -1) {
                int maxDistance = -1;
                for (int j = 0; j < frames; j++) {
                    int nextUse = -1; // Initialize to -1 to indicate the page won't be used in the future
                    for (int k = i + 1; k < pages; k++) {
```

```

        if (frame[j] == referenceString[k]) {
            nextUse = k; // Find the next occurrence of the page
            break;
        }
    }

    // If a page won't be used again (nextUse == -1), replace it immediately
    if (nextUse == -1) {
        index = j;
        break;
    }

    // Track the farthest used page
    if (nextUse > maxDistance) {
        maxDistance = nextUse;
        index = j;
    }
}

// Replace the page in the chosen frame
frame[index] = referenceString[i];
pageFaults++; // Increment page fault when a new page is loaded
}

// Display the current state of the frames
printf("%d: ", referenceString[i]);
for (int k = 0; k < frames; k++) {
    printf("%d ", frame[k]);
}
printf("\n");
}

// Output total page faults
printf("Total Page Faults: %d\n", pageFaults);
return 0;
}

```

Output:

```

Enter number of frames: 3
Enter number of pages: 7
Enter reference string: 1 2 3 1 4 2 1
1: 1 -1 -1
2: 1 2 -1
3: 1 2 3
1: 1 2 3
4: 1 2 4
2: 1 2 4
1: 1 2 4
Total Page Faults: 4

```

Lab Program 8:

Write a C program to simulate the following file allocation strategies.

- a) Sequential
- b) Indexed
- c) Linked

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX_BLOCKS 100

int memory[MAX_BLOCKS]; // 0 for free, 1 for allocated

void initialize_memory() {
    for (int i = 0; i < MAX_BLOCKS; i++)
        memory[i] = 0;
}

// Sequential Allocation
void sequential_allocation() {
    int start, length, i, flag = 0;

    printf("\nSequential Allocation\n");
    printf("Enter starting block and length of file: ");
    scanf("%d %d", &start, &length);

    if (start + length > MAX_BLOCKS) {
        printf("Allocation exceeds memory limit.\n");
        return;
    }

    for (i = start; i < start + length; i++) {
        if (memory[i] == 1) {
            flag = 1;
            break;
        }
    }

    if (flag == 0) {
        for (i = start; i < start + length; i++) {
            memory[i] = 1;
        }
        printf("File allocated from block %d to %d\n", start, start + length - 1);
    } else {
        printf("Block already allocated. Cannot allocate file.\n");
    }
}

// Indexed Allocation
void indexed_allocation() {
    int index, length, blocks[50];
```

```

printf("\nIndexed Allocation\n");
printf("Enter index block: ");
scanf("%d", &index);

if (memory[index] == 1) {
    printf("Index block already allocated.\n");
    return;
}

printf("Enter number of blocks and block numbers: ");
scanf("%d", &length);

for (int i = 0; i < length; i++) {
    scanf("%d", &blocks[i]);
    if (blocks[i] < 0 || blocks[i] >= MAX_BLOCKS || memory[blocks[i]] == 1) {
        printf("Block %d already allocated or invalid.\n", blocks[i]);
        return;
    }
}

memory[index] = 1;
for (int i = 0; i < length; i++) {
    memory[blocks[i]] = 1;
}

printf("File allocated with index block %d pointing to blocks: ", index);
for (int i = 0; i < length; i++) {
    printf("%d ", blocks[i]);
}
printf("\n");
}

// Linked Allocation
typedef struct Block {
    int blockNum;
    struct Block* next;
} Block;

void linked_allocation() {
    int length, blocks[50];

    printf("\nLinked Allocation\n");
    printf("Enter number of blocks: ");
    scanf("%d", &length);

    printf("Enter block numbers: ");
    for (int i = 0; i < length; i++) {
        scanf("%d", &blocks[i]);
        if (blocks[i] < 0 || blocks[i] >= MAX_BLOCKS || memory[blocks[i]] == 1) {
            printf("Block %d already allocated or invalid.\n", blocks[i]);
            return;
        }
    }
}

```

```

for (int i = 0; i < length; i++) {
    memory[blocks[i]] = 1;
}

Block* head = malloc(sizeof(Block));
head->blockNum = blocks[0];
Block* current = head;

for (int i = 1; i < length; i++) {
    current->next = malloc(sizeof(Block));
    current = current->next;
    current->blockNum = blocks[i];
}
current->next = NULL;

printf("File allocated using linked allocation. Blocks: ");
current = head;
while (current != NULL) {
    printf("%d -> ", current->blockNum);
    Block* temp = current;
    current = current->next;
    free(temp); // Freeing after printing
}
printf("NULL\n");
}

int main() {
    int choice;

    initialize_memory();

    do {
        printf("\nFile Allocation Strategies:\n");
        printf("1. Sequential Allocation\n");
        printf("2. Indexed Allocation\n");
        printf("3. Linked Allocation\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                sequential_allocation();
                break;
            case 2:
                indexed_allocation();
                break;
            case 3:
                linked_allocation();
                break;
            case 4:
                printf("Exiting program.\n");
                break;
            default:

```

```

        printf("Invalid choice.\n");
    }
} while (choice != 4);

return 0;
}

```

Output:

<pre> File Allocation Strategies: 1. Sequential Allocation 2. Indexed Allocation 3. Linked Allocation 4. Exit Enter your choice: 1 Sequential Allocation Enter starting block and length of file: 5 5 File allocated from block 5 to 9 </pre>	<pre> File Allocation Strategies: 1. Sequential Allocation 2. Indexed Allocation 3. Linked Allocation 4. Exit Enter your choice: 2 Indexed Allocation Enter index block: 10 Enter number of blocks and block numbers: 2 12 13 File allocated with index block 10 pointing to blocks: 12 13 </pre>
--	--

```

File Allocation Strategies:
1. Sequential Allocation
2. Indexed Allocation
3. Linked Allocation
4. Exit
Enter your choice: 3

Linked Allocation
Enter number of blocks: 3
Enter block numbers: 25 50 70
File allocated using linked allocation. Blocks: 25 -> 50 -> 70 ->
NULL

```